



**UNIVERSIDAD LATINA
DE COSTA RICA**

POWERED BY **Arizona State University**

UNIVERSIDAD LATINA DE COSTA RICA

Programación V

MANUAL TÉCNICO

Proyecto: ParchisOnline (Parchís)

Arquitectura, instalación y configuración del sistema

Profesor: Osvaldo Granados Venegas

Estudiante: Azzel Brenes Pereira

12 agosto, 2026

Índice

1. Introducción	3
2. Alcance del Manual.....	3
3. Arquitectura del Sistema.....	4
3.1 Visión general	4
3.2 Dirección de dependencias.....	5
3.3 Comunicación en tiempo real (SignalR).....	6
3.4 Motor del juego.....	6
4. Tecnologías y Herramientas.....	7
4.1 Backend (.NET 10)	7
4.2 Frontend (Ionic + Angular)	8
4.3 Herramientas de desarrollo	8
5. Requisitos Previos	9
6. Instalación y Configuración del Entorno.....	9
6.1 Obtener el código fuente	9
6.2 Backend — configuración en Visual Studio.....	10
6.3 Configuración de appsettings.json	10
6.4 Base de datos	11
6.5 Frontend — configuración en Visual Studio Code.....	11
7. Estructura del Proyecto.....	12
7.1 Backend (Parchis_G3)	12
7.2 Frontend (ParchisOnline_FrontEnd)	13
8. Base de Datos.....	14
8.1 Diagrama entidad-relación	14
8.2 Descripción de las tablas	15
9. Seguridad Implementada	17
10. Ejecución del Sistema	18
11. Resumen de Módulos Funcionales	19
12. Solución de Problemas Comunes.....	21
Anexo A. Requerimientos Funcionales (Resumen).....	22

1. Introducción

Este manual técnico documenta la arquitectura, instalación y configuración de ParchisOnline, una aplicación móvil de Parchís multijugador en línea desarrollada como proyecto del curso Programación V. El sistema permite a los usuarios registrarse, unirse a salas de apuesta con monedas virtuales, jugar partidas de Parchís en tiempo real contra otros jugadores o bots, personalizar su experiencia mediante una tienda de artículos, y consultar estadísticas, historial y rankings.

El proyecto está compuesto por dos partes principales: un backend construido en .NET 10 con arquitectura en capas, y una aplicación móvil híbrida construida con Ionic y Angular, empaquetada con Capacitor. La comunicación en tiempo real (turnos, movimientos de fichas, chat) se maneja mediante SignalR.

2. Alcance del Manual

Este documento cubre los siguientes aspectos del sistema:

1. La arquitectura técnica del backend (capas) y del frontend (Ionic/Angular).
2. Las tecnologías, librerías y herramientas utilizadas en el desarrollo.
3. Los requisitos previos y pasos de instalación del entorno de desarrollo, tanto en Visual Studio como en Visual Studio Code.
4. La configuración de la base de datos, la API y las variables sensibles del sistema.
5. Las medidas de seguridad implementadas.

6. Un resumen de los módulos funcionales del sistema, con base en el documento de requerimientos del proyecto.

No es un manual de usuario final; el enfoque es exclusivamente técnico, orientado a desarrolladores que necesiten instalar, configurar, mantener o extender el sistema.

3. Arquitectura del Sistema

3.1 Visión general

El sistema sigue una arquitectura cliente-servidor. El cliente es una aplicación móvil híbrida (Ionic + Angular + Capacitor) que consume una API REST y se conecta mediante WebSockets (SignalR) para todo lo que ocurre en tiempo real durante una partida. El servidor (backend) está organizado en 5 capas independientes, cada una en su propio proyecto de .NET, lo que separa responsabilidades y facilita el mantenimiento y las pruebas.

Capa / Proyecto	Responsabilidad
ParchisOnline_FrontEnd (Ionic + Angular + Capacitor)	Interfaz de usuario, navegación por páginas, consumo de la API REST y del Hub de SignalR.
Parchis_G3.API	Controladores REST, autenticación JWT, Hub de SignalR, middlewares (CORS, rate limiting, cabeceras de seguridad), Swagger.
Parchis_G3.LogicaNegocios	Reglas de negocio: motor del juego, matchmaking, bots, chat, abandono/reconexión, seguridad de cuentas.
Parchis_G3.AccesoDatos	DbContext de Entity Framework Core, Unidad de Trabajo, implementaciones de acceso a SQL Server.
Parchis_G3.Dominio	Entidades, DTOs e interfaces (contratos) compartidas entre las demás capas.
Parchis_G3.Utilitarios	Utilidades transversales: validadores de entrada, formato estándar de respuestas de la API.

3.2 Dirección de dependencias

La API depende de la capa de Lógica de Negocios y de Acceso a Datos; la Lógica de Negocios depende del Dominio y de Utilitarios; el Acceso a Datos depende del Dominio. El Dominio no depende de ninguna otra capa: contiene únicamente entidades, DTOs e interfaces, lo que permite que las demás capas se comuniquen entre sí por contrato (inyección de dependencias) y no por implementación concreta.

3.3 Comunicación en tiempo real (SignalR)

El estado de una partida (turnos, resultado del dado, movimiento de fichas, mensajes de chat) se transmite en vivo mediante un Hub de SignalR expuesto en la ruta /hubs/partida (clase PartidaHub). Dos particularidades importantes de esta configuración:

1. El Hub usa su propio serializador JSON, independiente del de los controladores REST. Se configura con `AddJsonProtocol` y `PropertyNamingPolicy = null` para que las propiedades viajen en `PascalCase`, tal como las espera el frontend Angular.
2. Los WebSockets no permiten enviar cabeceras personalizadas, por lo que el token JWT se envía como query string (`?access_token=...`) y se valida en el evento `OnMessageReceived` únicamente para las rutas que empiezan con /hubs.

3.4 Motor del juego

La lógica del tablero (movimiento de fichas, capturas, coronación, turnos dobles, límite de 30 segundos por turno) vive en `MotorParchisLN`, registrado como servicio Singleton porque mantiene en memoria el estado activo de cada partida (turno actual, racha de dobles, dado pendiente). Al ser Singleton, no puede depender directamente de servicios Scoped como el `DbContext`; en su lugar, la Unidad de Trabajo (`IUnidadTrabajoEF`) se recibe como parámetro en cada método que necesita persistir datos.

4. Tecnologías y Herramientas

4.1 Backend (.NET 10)

Tecnología	Uso en el proyecto
ASP.NET Core Web API (net10.0)	Framework base de la API REST.
Entity Framework Core 10 + SQL Server	Acceso a datos (ORM) contra SQL Server.
AutoMapper	Mapeo entre entidades y DTOs.
BCrypt.Net-Next	Hash seguro de contraseñas de usuario.
JWT Bearer Authentication	Autenticación basada en tokens.
SignalR	Comunicación en tiempo real durante la partida.
Newtonsoft.Json	Serialización JSON adicional (localización, compatibilidad).
NLog	Registro (logging) de la aplicación.
Swashbuckle (Swagger)	Documentación e interfaz de pruebas de la API.
PayPal Checkout SDK	Procesamiento de pagos en la tienda de monedas (modo sandbox).
Rate Limiting (ASP.NET Core)	Protección contra fuerza bruta y abuso de endpoints.

4.2 Frontend (Ionic + Angular)

Tecnología	Versión	Uso en el proyecto
Angular	20.3	Framework principal (standalone components).
Ionic Framework	8.x	Componentes de interfaz móvil.
Capacitor	8.x	Empaquetado nativo para Android/iOS.
@microsoft/signalr	10.x	Cliente para conectarse al Hub de SignalR.
RxJS	7.8	Programación reactiva (observables).
TypeScript	5.9	Lenguaje base del frontend.

4.3 Herramientas de desarrollo

1. Visual Studio 2022 (17.8 o superior, con soporte para .NET 10 preview) — desarrollo del backend.
2. Visual Studio Code con extensiones de Angular/Ionic — desarrollo del frontend.
3. SQL Server Management Studio (SSMS) o Azure Data Studio — administración de la base de datos.
4. Node.js (LTS) y npm — gestor de paquetes del frontend.
5. Ionic CLI — comandos de build, serve y empaquetado nativo.
6. Swagger UI — pruebas manuales de los endpoints de la API.

5. Requisitos Previos

Antes de instalar el proyecto, verificar que el equipo cuente con lo siguiente:

1. .NET 10 SDK (preview) — el proyecto apunta a net10.0 en los cuatro proyectos del backend.
2. SQL Server Express (o Developer/Standard) con una instancia accesible; el proyecto original usa una instancia llamada SQLEXPRESS.
3. Visual Studio 2022 con la carga de trabajo "Desarrollo de ASP.NET y web".
4. Visual Studio Code (para el frontend).
5. Node.js 18 LTS o superior, con npm.
6. Ionic CLI: `npm install -g @ionic/cli`
7. Git, para clonar el repositorio.
8. (Opcional) Android Studio, si se va a compilar/probar la app en un emulador o dispositivo Android mediante Capacitor.

6. Instalación y Configuración del Entorno

6.1 Obtener el código fuente

Clonar o descomprimir el repositorio del proyecto. La estructura contiene dos carpetas principales: Parchis_G3 (backend, .NET) y ParchisOnline_FrontEnd (frontend, Ionic/Angular), dentro de la carpeta del backend.

6.2 Backend — configuración en Visual Studio

1. Abrir el archivo de solución Parchis_G3.slnx con Visual Studio 2022.
2. Esperar a que Visual Studio restaure automáticamente los paquetes NuGet (o ejecutar dotnet restore desde la carpeta Parchis_G3).
3. Configurar la cadena de conexión y las claves sensibles en appsettings.json / appsettings.Development.json del proyecto Parchis_G3.API (ver sección 6.3).
4. Establecer Parchis_G3.API como proyecto de inicio.
5. Ejecutar con F5 (o Ctrl+F5 sin depurar). El backend queda escuchando en `http://localhost:5051`, con Swagger disponible en `/swagger`.

6.3 Configuración de appsettings.json

El proyecto usa tres bloques de configuración sensible que deben ajustarse a cada entorno de desarrollo. Por buenas prácticas, estos valores no deben publicarse en un repositorio público; se muestran aquí a modo de referencia de la estructura esperada, con valores de ejemplo:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=TU_SERVIDOR\\SQLEXPRESS;
      Initial Catalog=ParchisOnline;User ID=tu_usuario;
      Password=tu_password;TrustServerCertificate=true;"
  },
  "Jwt": {
    "Key": "clave-secreta-larga-y-unica-para-firmar-tokens"
  },
  "PayPal": {
    "ClientId": "tu-client-id-de-paypal-sandbox",
    "Secret": "tu-secret-de-paypal-sandbox",
    "Sandbox": true
  }
}
```

1. `ConnectionStrings:DefaultConnection` — apunta a la instancia de SQL Server donde se ejecutó el script de la base de datos (ver sección 8).
2. `Jwt:Key` — clave simétrica usada para firmar y validar los tokens de acceso; debe tener al menos 32 caracteres.
3. `PayPal:ClientId / Secret` — credenciales de una cuenta sandbox de PayPal Developer, usadas por la tienda de compra de monedas.

6.4 Base de datos

Ejecutar el script `ParchisOnline_BaseDeDatos.sql` contra la instancia de SQL Server (por ejemplo desde SSMS o Azure Data Studio). El script crea la base de datos ParchisOnline y sus tablas. El detalle de la estructura se documenta en la sección 8.

6.5 Frontend — configuración en Visual Studio Code

1. Abrir la carpeta `ParchisOnline_FrontEnd/ParchisOnline_FrontEnd` en Visual Studio Code.
2. Instalar las dependencias: `npm install`
3. Verificar que `src/app/services/conexion.ts` apunte a la URL del backend (por defecto `http://localhost:5051/api`).
4. Ejecutar en modo desarrollo (navegador): `ionic serve`, o `npm start`.
5. Para probar en Android: `ionic capacitor add android`, seguido de `ionic capacitor sync android` y `ionic capacitor open android`.

El backend debe estar corriendo antes de iniciar el frontend, ya que la app valida la disponibilidad de la API al arrancar y muestra un aviso si no logra conectarse.

7. Estructura del Proyecto

7.1 Backend (Parchis_G3)

Parchis_G3/

- | — Parchis_G3.API/ → Controllers, Hubs, Services, Program.cs
- | | — Controllers/ → 23 controladores REST (Usuario, Sala, Partida, ...)
- | | — Hubs/PartidaHub.cs → Hub de SignalR para tiempo real
- | | — Services/
- | — Parchis_G3.LogicaNegocios/
- | | — Implementaciones/ → Reglas de negocio por módulo
- | | — Motor/ → Motor del juego (MotorParchisLN)
- | — Parchis_G3.AccesoDatos/
- | | — Contexto/ParchisOnlineContext.cs
- | | — Implementaciones/ → Unidad de Trabajo, repositorios
- | — Parchis_G3.Dominio/
- | | — Entidades/ DTO/ EntidadesTipadas/
- | | — InterfacesAD/ InterfacesLN/
- | — Parchis_G3.Utilitarios/ → Respuesta.cs, ValidadorInput.cs

7.2 Frontend (ParchisOnline_FrontEnd)

ParchisOnline_FrontEnd/src/app/

- |— pages/
 - | |— login/ registro/ home/ tablero/ sala-espera/
 - | |— tienda/ tienda-monedas/ ranking/ logros/
 - | |— perfil/ configuración/ tutorial/
- |— services/
 - |— auth.ts conexion.ts signalr.service.ts
 - |— sala.service.ts partida.service.ts inventario.service.ts
 - |— pago.service.ts perfil.service.ts ranking.service.ts
 - |— recompensa.service.ts tienda.service.ts inactividad.ts

Cada página del frontend corresponde a una pantalla de la aplicación (login, tablero de juego, tienda, ranking, etc.) y se apoya en los servicios correspondientes para comunicarse con la API REST o el Hub de SignalR.

8.2 Descripción de las tablas

Grupo	Tabla	Descripción
Cuentas y seguridad	Usuarios	Datos de cuenta, saldo de monedas, estadísticas y configuración del jugador.
	SesionesActivas	Sesiones abiertas por usuario; soporta reconexión y cierre automático por inactividad.
	SegLogs	Bitácora de seguridad: intentos de acceso, bloqueos de cuenta.
Partidas y juego	Salas	Salas de apuesta disponibles, con su costo de entrada.
	Partidas	Partidas creadas a partir de una sala, en curso o finalizadas.
	JugadoresPartida	Relación entre un usuario y una partida; incluye sus fichas y color.
	EstadoFichas	Posición actual de cada ficha en el tablero.
	TurnosPartida	Historial de turnos jugados: resultado del dado y acción tomada.
	FilaEspera	Cola de jugadores que ingresan a una sala cuya partida ya inició.
	HistorialPartidas	Resultado final (victoria/derrota) de cada partida por usuario.

Grupo	Tabla	Descripción
Economía y tienda	Articulos	Fichas, tableros y dados disponibles en la tienda de personalización.
	TiposArticulo	Categoría de cada artículo (ficha, tablero, dado).
	UsuarioArticulos	Artículos que un usuario ha comprado/desbloqueado.
	EquipamientoActivo	Artículo actualmente equipado por el usuario en cada categoría.
	Transacciones	Historial de movimientos de monedas (entradas, premios, compras).
Comunicación	MensajesChat	Mensajes de chat enviados durante cada partida.

9. Seguridad Implementada

El backend incorpora varias capas de protección, configuradas principalmente en Program.cs:

1. Autenticación con JWT: cada request protegido requiere un token válido firmado con la clave configurada en Jwt:Key.
2. Contraseñas con hash BCrypt: las contraseñas nunca se almacenan en texto plano.
3. Bloqueo de cuenta persistente: los intentos fallidos de acceso se registran en base de datos (no en memoria), por lo que un reinicio del servidor no desbloquea a un atacante.
4. Rate limiting por IP: 5 intentos de login por minuto, 3 registros de cuenta por hora, y un límite global de 100 solicitudes por minuto (los WebSockets de SignalR quedan exentos, ya que una partida genera tráfico legítimo constante).
5. Cabeceras de seguridad HTTP en todas las respuestas: X-Content-Type-Options, X-Frame-Options, Referrer-Policy y Permissions-Policy.
6. CORS restringido a los orígenes conocidos de la app móvil (localhost e Ionic/Capacitor), con credenciales habilitadas.
7. Cierre automático de sesión tras 30 minutos de inactividad fuera de una partida, con aviso previo de 60 segundos (RF-20).

10. Ejecución del Sistema

Orden recomendado para levantar el proyecto en un entorno de desarrollo local:

1. Iniciar el servicio de SQL Server y confirmar que la base de datos ParchisOnline existe (sección 6.4).
2. Abrir Parchis_G3.slnx en Visual Studio y ejecutar el proyecto Parchis_G3.API (F5). Verificar que Swagger cargue correctamente en <http://localhost:5051/swagger>.
3. Abrir la carpeta del frontend en Visual Studio Code y ejecutar npm install (solo la primera vez).
4. Ejecutar ionic serve para levantar la app en el navegador, apuntando al backend activo.
5. Registrar un usuario de prueba desde la app o mediante Swagger, para validar el flujo de autenticación de punta a punta.
6. Abrir una segunda sesión (otro navegador o pestaña de incógnito) para simular una partida entre dos jugadores y confirmar que el Hub de SignalR sincroniza el tablero en tiempo real.

11. Resumen de Módulos Funcionales

A continuación, un resumen de los módulos del sistema, con base en el documento de requerimientos del proyecto (detalle completo en el Anexo A):

Módulo	Funcionalidad clave
Usuarios	Registro, inicio de sesión y saldo de monedas (5,000 de bienvenida).
Matchmaking y salas	5 salas de apuesta; inicio automático con 2+ jugadores completando con bots.
Motor del juego	Reglas oficiales del Parchís: dado, capturas, coronación, turnos y límite de tiempo.
Bots	Completan turnos automáticamente con lógica de captura y avance.
Economía virtual	Descuento de entrada, premios, historial de transacciones.
Tienda de personalización	Fichas, tableros y dados comprables con monedas virtuales.
Monedas diarias	Recompensa por racha de ingreso (200 a 1,000 monedas).
Compra de monedas	Paquetes de monedas pagados con dinero real vía pasarela de pago.
Chat de partida	Mensajes predefinidos y libres, con cooldown anti-spam de 5s.
Historial y estadísticas	Partidas jugadas/ganadas/perdidas y porcentaje de victoria.
Notificaciones push	Aviso de turno, partida lista y monedas acreditadas.
Ranking	Tabla de posiciones global por monedas ganadas en partidas.

Módulo	Funcionalidad clave
Reconexión	Reserva de lugar por 60s ante pérdida de conexión.
Abandono	Penalización del 20% de la entrada y bloqueo tras 3 abandonos seguidos.
Configuración de cuenta	Cambio de contraseña, notificaciones, sonido y cierre de sesión.
Tutorial interactivo	Guía inicial para nuevos usuarios, disponible desde el menú de ayuda.
Sonido y animaciones	Efectos de dado, movimiento, captura y coronación.
Modo sin conexión	Mensajes claros ante falta de red o mantenimiento del servidor.
Cierre automático de sesión	Por inactividad de 30 minutos, con aviso previo de 60 segundos.

12. Solución de Problemas Comunes

Problema	Causa probable	Solución
La API no conecta a la base de datos	Cadena de conexión incorrecta o el servicio de SQL Server no está iniciado	Verificar el servicio de SQL Server y el valor de <code>ConnectionStrings:DefaultConnection</code> en <code>appsettings.json</code>
El frontend recibe error de CORS	El origen desde el que corre la app no está en la política <code>PoliticaAndroid</code>	Agregar el origen (por ejemplo, el puerto usado por ionic serve) a la lista de orígenes permitidos en <code>Program.cs</code>
El tablero se dibuja vacío o sin fichas	El Hub de SignalR no tiene configurado <code>PropertyNamingPolicy = null</code> y envía las propiedades en <code>camelCase</code>	Confirmar que <code>AddSignalR().AddJsonProtocol(...)</code> esté presente en <code>Program.cs</code>
"401 Unauthorized" al usar la app	El token JWT expiró, o <code>Jwt:Key</code> no coincide con el valor usado al generarlo	Volver a iniciar sesión; verificar que <code>Jwt:Key</code> sea el mismo en todo el entorno
Nombres de tabla inesperados al migrar con EF Core	Entity Framework pluraliza automáticamente los nombres de las entidades	Mapear explícitamente cada tabla con <code>ToTable("NombreExacto")</code> dentro de <code>OnModelCreating</code>

Problema	Causa probable	Solución
"429 Too Many Requests"	Se superó el límite de rate limiting (5/min en login, 100/min global)	Esperar la ventana de tiempo indicada; revisar los límites configurados en Program.cs si es demasiado estricto para pruebas

Anexo A. Requerimientos Funcionales (Resumen)

Resumen de los 20 requerimientos funcionales definidos para el proyecto ParchisOnline.

El detalle completo de cada uno se encuentra en el documento de requerimientos del curso.

Código	Nombre	Resumen
RF-01	Gestión de Usuarios	Registro, login y saldo de monedas de bienvenida.
RF-02	Matchmaking y Salas	5 salas de apuesta; inicio automático completando con bots.
RF-03	Motor del Juego	Reglas oficiales del Parchís, turnos y límite de 30s.
RF-04	Sistema de Bots	Turnos automáticos con lógica de captura y avance.
RF-05	Economía Virtual	Descuento de entrada, premios y transacciones.
RF-06	Tienda de Personalización	Fichas, tableros y dados comprables con monedas.
RF-07	Monedas Diarias	Recompensa por racha de ingreso diario.

Código	Nombre	Resumen
RF-08	Tienda de Compra de Monedas	Paquetes de monedas pagados con dinero real.
RF-09	Mensajes Predefinidos y Chat	Chat de partida con cooldown anti-spam.
RF-10	Historial y Estadísticas	Partidas jugadas/ganadas y porcentaje de victoria.
RF-11	Notificaciones Push	Aviso de turno, partida lista y monedas acreditadas.
RF-12	Ranking	Tabla de posiciones global por monedas ganadas.
RF-13	Reconexión a Partida	Reserva de lugar por 60 segundos.
RF-14	Abandono de Partida	Penalización del 20% y bloqueo tras 3 abandonos.
RF-15	Configuración de Cuenta	Contraseña, notificaciones, sonido y cierre de sesión.
RF-16	Tutorial Interactivo	Guía inicial para nuevos usuarios.
RF-17	Sonidos y Música	Efectos de dado, movimiento, captura y coronación.
RF-18	Animaciones del Tablero	Movimiento animado de fichas y celebraciones.
RF-19	Modo Mantenimiento y Sin Conexión	Mensajes claros ante falta de red o mantenimiento.
RF-20	Cierre Automático de Sesión	Cierre por inactividad de 30 minutos.